

Confluence and Fork-Safety in Event-Sourced Agent Runtimes

Yohei Nakajima

Research draft — August 2026

Abstract

Event-sourced agent runtimes commonly claim deterministic replay, cheap forking, and end-to-end lineage. These claims are attractive but incomplete. A pure reducer is deterministic only after the event sequence is fixed; a reactive runtime may admit multiple firing orders; an oracle result must be captured before it can be replayed; and an event-log fork cannot undo an email, charge, publication, or model-call cost that already occurred. This paper asks when replay and fork claims are sound, rather than proposing another minimal agent primitive.

We present an executable F# specification that separates replayable facts from requested effects, exposes scheduler order as a parameter, and assesses fork safety relative to four distinct properties: retained-prefix projection, strict execution replay, continuation in a prefix-reconstructed environment, and equivalence to a counterfactual external world. Effects are described by three orthogonal dimensions: external footprint, replay source, and lifecycle. FsCheck laws and preserved counterexamples show that settlement is neither terminating nor confluent in general. Overlapping writers yield different normal forms, while a second counterexample shows that disjoint declared writes remain insufficient when behavior reads and emitted triggers interfere.

We validate the conditions against two runtime artifacts comprising 5,564 run logs and 314,348 source events. In a public Synthetic Players capsule, all 317,296 projection cuts are sound under the stated projection observation, but 36,251 cuts split an outstanding model request and are unsound for external continuation. The store contains 121 observed forks; their 16,625 shared-prefix events have zero structural mismatches modulo child run identity and global database sequence, and all observed cuts avoid external requests. A separate 24-run offline bridge study has 66.7% decision agreement but 0% ordered-path agreement, illustrating that replay, path, projection, and decision equivalence are different assertions. The results turn “cheap fork” from a performance slogan into a property-indexed semantic claim with checkable preconditions.

Artifact status. The code, law suite, conformance vectors, and evidence ledger accompany this draft. The public capsule is revision- and hash-pinned. The bridge artifact is locally hash-bound but its containing source directory has no commit or remote; we therefore treat it as non-reproducible outside the local environment until published.

1 Introduction

Agent runtimes increasingly retain an append-only record of model calls, tool results, human decisions, state updates, and coordination events. Event sourcing makes several desirable properties seem immediate. If state is a fold over the log, a run can be replayed. If a child log shares a prefix with a parent, a run can be forked without recomputing that prefix. If causes are recorded, artifacts can be traced back to model and tool activity. ActiveGraph states these properties as consequences of making the log the source of truth [8].

This paper formalizes and empirically checks the replay and fork properties claimed on architectural grounds in Nakajima (2026). The relationship is deliberate: the earlier work describes a deployed architecture, while the present work asks which side conditions make its architectural claims sound. Negative results about the originating runtime are therefore part of the contribution rather than exceptions to be hidden.

Two gaps motivate the work. First, “deterministic replay” often means deterministic replay *after oracle outputs have been fixed*. This is useful, but the phrase suppresses operational questions. Was the model output captured faithfully? Is a request retained without its result? Does replay serve the recorded result or make a new call? Is the runtime itself schedule-independent, or does a particular FIFO order merely produce one repeatable execution?

Second, event sourcing in a business application normally reconstructs application state; it does not claim to reverse the external world. That distinction becomes load-bearing for agents. A retrospectively discarded log suffix may contain an email, a charge, a delete, a publication, or a billable model call. The forked trace can omit the effect, but the world cannot be assumed to be one in which the effect never happened. A correct fork contract must say which property it preserves.

The initial version of this project asked whether

$$A : S \times E \rightarrow S \times F^* \tag{1}$$

was the irreducible agent primitive. That framing is not novel. The shape is a Mealy-style transition system [7], closely resembles the Elm update architecture [3], and appears in functional event sourcing through the Decider pattern [2]. We concede the minimality lane and instead exploit the effects/events seam that becomes visible when agents interact with irreversible external systems.

Contributions. This draft makes four contributions.

1. It gives an executable operational model in which scheduler order is explicit. The model separates termination, blocking on an external effect, and bounded divergence.
2. It preserves counterexamples showing that general settlement is neither terminating nor confluent. It further shows that pairwise-disjoint *declared writes* are insufficient when behavior reads and emitted triggers interfere. A restricted isolated-writer family does converge, but we do not claim a minimal general condition.
3. It defines property-relative fork assessment over three orthogonal effect dimensions: footprint, replay source, and lifecycle. This separates exact retained-prefix projection from execution replay, continuation, and a counterfactual-world claim.
4. It checks the laws against 5,564 runtime logs, 314,348 source events, every possible cut in each log, and 121 observed forks. The validation finds both a strong positive result—zero structural mismatches in the observed shared prefixes—and sharp limitations on which hypothetical cuts license continuation or counterfactual claims.

Genre. This is not a mechanized-proof paper. Laws are stated as propositions, checked over generated families with FsCheck, and evaluated exhaustively over the cuts of the available logs. A failed law produces a minimized, named regression fixture. Passing checks are finite evidence over the specified generators and artifacts, not universal proofs. This choice is intentional: calculi with machine-checked composition, termination, and information-flow results already exist [5, 4]. Our comparative advantage is an executable runtime contract connected to real runtime artifacts.

Roadmap. Section 2 defines the executable model and observations. Section 3 reports termination and confluence results. Section 4 defines fork properties and effect conditions. Section 5 connects those conditions to replayability grades. Sections 6 and 7 describe the artifact and empirical validation. We then position the work, discuss implications, and state limitations.

2 Executable Runtime Model

The model is intentionally small, but it is not a claim of expressive minimality. Its purpose is to make replay, scheduling, and effects observable enough to falsify laws.

2.1 Events, projection, and identity

An event envelope is a tuple

$$e = (id, run, seq, kind, cause, emitter). \quad (2)$$

The event kind is a closed sum containing domain facts, signals, effect lifecycle events, and evidence facts. A log $L = \langle e_1, \dots, e_n \rangle$ is ordered and append-only. The pure reducer $evolve : S \times E \rightarrow S$ yields

$$\text{project}(L) = \text{fold}(\text{evolve}, S_0, L). \quad (3)$$

Event identity is idempotent at the reducer boundary: if an event identifier has already been applied, a repeated envelope does not mutate state again. A duplicate identifier remains an integrity fault for fork assessment, because idempotent reduction must not silently certify a malformed trace.

State contains domain facts and signals, projected effect descriptors, evidence facts, integrity faults, and the set of applied event identifiers. When comparing projected domain state, the implementation excludes the bookkeeping set of applied identifiers. This prevents a replay that merely renames envelopes from being declared semantically different while still allowing exact-trace equality to detect the change.

Proposition 1 (Closed-log projection determinism). *For a fixed ordered event log and a pure reducer, repeated evaluation of Equation 3 returns the same state. Oracle calls are not an exception: their recorded results, when present, are events in the fixed log.*

This proposition is the trivial base case. It says nothing about how the log was produced, whether the event order was unique, whether missing oracle results can be recovered, or whether effects may be executed again.

2.2 Behaviors and actions

A behavior is a tuple

$$b = (id_b, trigger_b, fire_b, W_b) \quad (4)$$

where $trigger_b : S \times E \rightarrow \mathbb{B}$ decides whether an event enables the behavior, $fire_b : S \times E \rightarrow A^*$ returns actions, and W_b is a declared write set used only as a diagnostic. Actions can emit a domain fact or signal, or request an external effect. Behaviors cannot emit an effect outcome or evidence attestation directly; those facts must cross the interpreter or validation boundary. This prevents a reducer from minting its own proof that an external action succeeded or that a run was verified.

The kernel does not execute actions. Emissions become pending events. Effect requests become pending request events and later block until an interpreter injects a committed, failed, or unknown outcome. Ordinary .NET tasks, network clients, mutable caches, and database adapters may exist outside the boundary; none are reachable from *trigger* or *fire*.

2.3 Configurations and scheduling

A configuration contains projected state, pending events, enabled activations, outstanding effects, the retained trace, the behavior firing trace, and the next sequence and generated-identity counters. A scheduler has two explicit selectors: one chooses a pending event, and the other chooses an enabled activation. Four policies are included: canonical, reverse-event, reverse-activation, and both reversed.

Processing a pending event first applies it to state and then computes enabled behaviors from the updated state. Enabled behaviors fire sequentially. A later activation handling the same triggering event therefore observes state changes produced by earlier activations. This choice mirrors the ordering seam found in `ActiveGraph` and makes interference visible.

Let $C \rightarrow_{\sigma} C'$ denote one legal event-processing or activation-firing step under scheduler σ . Settlement repeatedly steps until one of three outcomes is reached:

- **Quiescent**: no pending events, enabled activations, or outstanding effects;
- **BlockedAwaitingEffect**: no internal work remains, but at least one request has no terminal outcome;
- **Diverged**: a caller-supplied step bound is reached before either terminal outcome.

The step bound is checked only after testing for a terminal configuration. A run that becomes quiescent on the exact final step is therefore not misclassified as divergence. Conversely, reaching the bound does not prove a cycle; the kernel reports `StepBoundReached` and makes no stronger claim.

2.4 Observations and equivalence

Confluence and fork laws are meaningless without specifying what is observed. The artifact implements three equivalences:

1. **ExactTrace**: envelope and event lists are structurally equal;
2. **ProjectedState**: domain projection is equal after excluding applied-event bookkeeping and envelope metadata;
3. **FactsOnly(K)**: the selected fact keys agree.

The validation additionally reports byte equality when a source artifact provides it, decision equality when the study records decisions, and structural prefix equality modulo explicitly excluded lineage fields. These relations must not be substituted for one another. In particular, equal decisions do not imply equal paths, and equal projected state does not imply an unchanged external world.

3 Confluence and Quiescence

The ActiveGraph architecture describes behaviors reacting until quiescence without a privileged orchestrator. The working graph is then described as a deterministic projection of the log. Projection of a *fixed* log is deterministic, but the reactive loop can produce different logs under different legal schedules. We separate termination from uniqueness of the terminal projection.

Definition 1 (Termination at a bound). *For behavior set B , initial configuration C_0 , scheduler σ , and bound m , $\text{settle}_m(B, C_0, \sigma)$ terminates if it reaches Quiescent or BlockedAwaitingEffect in at most m steps. StepBoundReached is evidence of bounded nontermination, not a proof of infinite reduction.*

Definition 2 (Schedule confluence under an observation). *A behavior set is confluent for C_0 and observation P when every legal scheduler execution that reaches a quiescent configuration reaches a unique normal form modulo $\simeq_P$. Formally, if $C_0 \rightarrow_{\sigma_1}^* C_1$ and $C_0 \rightarrow_{\sigma_2}^* C_2$, and both C_1 and C_2 are quiescent, then $C_1 \simeq_P C_2$.*

This is Church–Rosser-shaped, but the current artifact does not prove a Church–Rosser theorem. It searches scheduler policies, generates restricted families, and preserves witnesses when the proposition fails.

3.1 A1: general termination is false

Counterexample 1 (Self-triggering emission). *Let a behavior trigger on signal `loop` and emit the same signal. Seed the pending queue with `loop`. Every processed signal enables the behavior, and every firing enqueues another signal. No quiescent or effect-blocked configuration is reached. For every generated bound from 1 through 100, the artifact reports StepBoundReached at that bound.*

The example is intentionally small. It shows that purity of the reducer and append-only logging do not imply termination of a reactive runtime. Detecting a repeated finite fingerprint would not solve the general problem: generated identities and sequence counters can make configurations syntactically fresh while their domain behavior cycles. The implementation therefore avoids a heuristic `CycleDetected` verdict.

Two natural restoring contracts remain candidates. One assigns every internally emitted event a well-founded rank that strictly decreases along trigger edges. The other requires an acyclic graph from event kinds to behavior emissions. Both would exclude the fixture, but neither is claimed minimal, and both need refinement for state-dependent triggers.

3.2 A2: general confluence is false

Counterexample 2 (Overlapping writers). *Behaviors α and β are both enabled by `start`. Each emits a write to fact `winner`; α writes 1 and β writes 2. Under canonical activation order the emitted events are processed so that the terminal winner is 2. Reversing activation order yields 1. Both executions quiesce, but their projected states differ.*

FsCheck generalizes this witness over generated integer values by constructing distinct adjacent values. Across the generated family, canonical and reverse activation schedules always disagree. The write-set diagnostic correctly reports `winner` as a conflict.

It is tempting to restore confluence by requiring declared write sets to be pairwise disjoint. That condition works for a deliberately restricted family: single-trigger behaviors that each unconditionally write a distinct key and do not read other behaviors’ regions or emit cross-triggering signals.

The suite generates up to eight such writers and checks all four scheduler policies; their projected states agree.

Declared disjointness is not sufficient in general.

Counterexample 3 (Read/trigger interference with disjoint writes). *Two behaviors, `left` and `right`, trigger on `start` and emit signals `left-ready` and `right-ready`. Their declared write sets are empty. A third behavior triggers on `left-ready`, reads whether `right-ready` is already in state, and writes the only fact `observed-right`. All declared write sets are disjoint. Under canonical event order the observer records 0; under reverse event order it records 1.*

The second witness matters because a write-write conflict detector would certify it. The actual dependency crosses four surfaces: an emission, an event selection, trigger eligibility, and a state read. Any useful general restoring condition must constrain at least behavior reads, behavior writes, emitted event kinds, and the trigger relation. Under-declared dependencies must either be rejected or made observable by instrumentation.

3.3 Production ordering masks the question

The ActiveGraph v1.10.0 scheduler uses a FIFO queue, evaluates matching behaviors in registration order, and dispatches them sequentially. Work due at the same delayed tick remains FIFO. This supplies a reproducible *ProductionOrder*, and it may be entirely appropriate as an operational contract. It is not evidence that alternate legal orders have the same normal form.

The distinction is practical. Projection occurs before scheduler notification, so an earlier registered behavior can emit an event whose projection changes the state read by a later behavior handling the same original event. Relation enumeration introduces another ordering seam when a backend query does not specify a canonical order. A stable FIFO queue can hide both effects in ordinary replay tests. Our scheduler tests bypass the production policy and vary event and activation order directly.

Result. The strongest supported conclusion is negative and conditional. General termination and general confluence are false. Isolated disjoint writers converge for the checked family, but declared writes alone do not restore confluence. Deriving and validating a general non-interference contract is future work; this draft does not present a minimal side condition.

4 Property-Relative Fork Safety

A trace fork at cut k retains $L_{<k}$ and gives subsequent events a child run identity. For a list representation,

$$\text{fork}(L, k) = \text{prefix}(L, k). \quad (5)$$

That definition makes prefix equality cheap, but it does not by itself make re-execution or external-world claims sound. We first describe effects, then define the properties a fork may be asked to preserve.

4.1 Effects along three dimensions

The initial research brief proposed a flat partition into Pure, Idempotent, Cached, and OneShot. Real traces show that the categories answer different questions. A cached model response describes

where replay material comes from; it does not describe whether the original call had external cost. We therefore use three orthogonal dimensions (Table 1).

Dimension	Values	Question
External footprint	Pure, Idempotent, Compensatable, OneShot, Unknown	What did execution do to the external world, and can that footprint be reconciled?
Replay source	Deterministic, Recorded, Uncaptured	Can replay reproduce or serve the result without consulting the live oracle?
Lifecycle	Requested, Committed, Failed, Unknown	Is the request/outcome boundary complete and trustworthy at this cut?

Table 1: Orthogonal effect dimensions.

Pure effects are internal descriptions with no external interaction. Idempotent effects can be repeated or overwritten under an explicit key. Compensatable effects require a separately verified compensation. OneShot effects are irreversible or conservatively treated as non-repeatable. Unknown fails closed. In the validation adapter, model calls are conservatively OneShot with respect to cost and oracle interaction. Their outputs may simultaneously be Recorded, allowing result replay without another call.

Lifecycle is projected from events. A request must precede exactly one terminal outcome. Duplicate requests, conflicting descriptors, an outcome without a request, or a second outcome after terminal state are integrity faults. The first valid request and first valid terminal outcome remain the projected authority; malformed later events do not overwrite them.

4.2 Four fork properties

Definition 3 (ProjectionReplay). *The child reconstructs the same domain projection as an independently folded copy of the retained, well-formed, ordered prefix. No execution occurs.*

Definition 4 (StrictExecutionReplay). *Every retained effect result needed by execution is deterministic or recorded, and the cut does not retain a Requested or Unknown lifecycle. Replaying the prefix causes zero live oracle or tool calls.*

Definition 5 (ExternalContinuation). *The target environment is assumed to represent the retained prefix. The child may continue after strict replay, provided inherited committed OneShot effects are served from the record and never executed again, and no inherited footprint is unknown. This property deliberately does not assert that a retrospectively discarded source suffix left the original world unchanged.*

Definition 6 (CounterfactualWorld). *In addition to ExternalContinuation, the actual external world is equivalent to one in which the discarded suffix did not occur. Discarded committed OneShot or unknown effects block the claim; discarded idempotent or compensatable effects make it conditional on reconciliation or successful compensation.*

These definitions make “sound” property-indexed. The same cut may be Sound for ProjectionReplay, Conditional for ExternalContinuation, and Unsound for CounterfactualWorld. Table 2 summarizes the executable assessment.

Property	Retained-prefix conditions	Discarded-suffix conditions
ProjectionReplay	Whole source log is structurally well formed; ordering authority is fixed	None for projected prefix equality
StrictExecutionReplay	No open/unknown lifecycle; every needed result is deterministic or recorded	None
ExternalContinuation	Strict replay; known footprint; retained committed OneShot effects are not re-executed	Target environment is defined to represent the prefix
CounterfactualWorld	ExternalContinuation conditions	No unreconciled external footprint; OneShot or unknown blocks

Table 2: Property-relative fork conditions.

4.3 Fork laws

The executable suite checks the following trace laws over generated logs.

Proposition 2 (Identity fork). *fork($L, |L|$) retains the full trace. Under ExactTrace, the shared prefix is equal to L ; child lineage is stored outside that shared list.*

Proposition 3 (Nested prefix collapse). *For $j \leq i \leq |L|$, retaining i events and then j events has the same shared prefix as retaining j events directly. The nested branch still records the outer child as its parent, so raw branch records are not identical.*

Proposition 4 (Projection commutation). *For a well-formed ordered log,*

$$\text{project}(\text{fork}(L, k)) = \text{project}(\text{prefix}(L, k)). \quad (6)$$

This is a trace/projection statement, not an external-world theorem.

Proposition 5 (Suffix irrelevance is property-relative). *The discarded suffix is irrelevant to ProjectionReplay of the retained prefix. It is not generally irrelevant to CounterfactualWorld.*

The last proposition corrects an over-broad formulation of cheap forking. A committed OneShot event in the discarded suffix is a concrete witness: the forked log excludes it and projects correctly, but the action remains true in the world. Likewise, a discarded request without a trustworthy terminal outcome is Unsound because the external system may have executed it even though the log never learned the result.

4.4 Cuts through effects

For each log, the validator constructs every prefix state in one forward scan and every environmental suffix finding in one reverse scan. It then assesses all $|L|+1$ cuts. Three boundary cases are especially important.

Before a request. The retained prefix does not inherit the effect. Projection replay is sound. If the source later committed a OneShot effect, CounterfactualWorld is unsound for a retrospective fork because that discarded action already occurred.

Between request and outcome. The prefix contains a Requested lifecycle. StrictExecution-Replay and ExternalContinuation are unsound: the effect may be running, may have committed without acknowledgement, or may have failed. A step bound or missing response must not be treated as a safe cancellation.

After a committed OneShot. The response may be Recorded, making strict replay possible with zero external calls. ExternalContinuation is Conditional: the interpreter must serve the recorded result and enforce zero re-executions. This is a semantic obligation, not merely an optimization.

These cases expose why a Decider that returns only domain events cannot express the full agent-runtime question. The log needs request/outcome boundaries, footprint evidence, and replay-source evidence before it can license execution or world-level claims.

5 Replayability Grades as Evidence Licenses

The activegraph-bridge project introduced five replayability levels from observed through native. The names arose from implementation failure modes. We make their meaning explicit by assigning each grade a set of licensed questions (Table 3).

Grade	Required evidence	Licensed use
Observed	Some inspectable trace or result	Inspection and available lineage only
Envelope	Faithful invocation envelope and completion, no lossy-envelope fact	Recorded-output playback; fork before invocation
Boundary	Envelope; mediated effects; clean reconstruction; complete known effect boundaries; no blockers	Re-execution using recorded effects; forks at effect boundaries
Checkpointed	Boundary plus recorded checkpoint	Resume from a restorable snapshot
Native	Checkpointed plus native-runtime evidence	Native replay, fork, and structural diff

Table 3: Replay grades and their licenses.

Let $Q(g)$ be the set of questions licensed by grade g . The current taxonomy is defined so that

$$Q(\text{Observed}) \subset Q(\text{Envelope}) \subset Q(\text{Boundary}) \subset Q(\text{Checkpointed}) \subset Q(\text{Native}). \quad (7)$$

Set inclusion supplies meet and join, so the five grades form a finite chain and therefore a lattice. The implementation checks idempotence, commutativity, absorption, and lower/upper-bound laws over every pair of grades. This result is definitional rather than a discovery about all possible taxonomies. A future capability taxonomy could contain incomparable evidence sets; the code’s comparison function refuses incomparable grades rather than inventing an order.

5.1 Computation from the log

The grade is computed from projected evidence and effect completeness. It is not accepted as a source label. In particular, a `NativeRuntime` marker alone does not grant Native: the log must also satisfy checkpoint, boundary, and envelope prerequisites. Unknown footprint, uncaptured results,

incomplete or unknown lifecycles, integrity faults, lossy envelopes, unmediated effects, and explicit hazards are blockers.

The adapter follows the same fail-closed rule. It grants evidence only for exact recognized source event types. An event whose name merely contains **verification** remains an unclassified domain signal. A verification event must contain an explicit Boolean verdict. The bridge-specific Boundary derivation additionally requires an effects-served count and an explicit null divergence. These requirements prevent an event-name coincidence from raising the grade.

5.2 Grades are not monotone in time

It is natural to view grades as achievements that only rise. That view is incorrect for evidence predicates.

Counterexample 4 (Hazard downgrade). *A prefix records a faithful envelope, completion, mediated boundary, clean reconstruction, and successful verification. It grades Boundary. Appending a hazard event adds a blocker, so the extended log grades Envelope.*

The grade of a *fixed log or prefix* licenses questions about that evidence. It need not be monotone under extension. This matters for forks: the grade of the shared prefix can be higher or lower than the grade of the completed source run, and a later hazard must not retroactively be ignored.

5.3 Connection to fork safety

The grade is a compact mechanism for checking some fork preconditions, but it does not replace property-relative assessment. Boundary evidence suggests that effect results and reconstruction are available, licensing forks at known effect boundaries. A specific cut can still split a request from its outcome. Likewise, a Boundary run can contain a committed OneShot effect that makes a CounterfactualWorld claim conditional or unsound.

The validation therefore reports both the run-level grade and all cut-level verdicts. A grade answers “what kinds of questions may this evidence support in principle?” A fork assessment answers “does this particular cut satisfy the conditions for property *P*?”

6 Executable Specification and Method

The artifact is a single .NET solution written in F#. Purity is enforced at the transition boundary, not imposed on the surrounding program. The source contains ordinary file I/O, JSON parsing, hashing, mutable library internals, and command-line orchestration outside the reducer. There is no database, dependency-injection container, actor framework, plugin system, network client, or asynchronous interpreter inside the kernel.

6.1 Artifact structure

The transition and settlement boundary consists of 594 source lines in `Domain.fs` and `Kernel.fs`. Fork and grade analyzers add 799 lines, for 1,393 core-law lines. Validation and conformance adapters are separate. This exceeds the initial 1,000-line whole-kernel target. We report the miss explicitly rather than compressing away lifecycle integrity, property-relative findings, or lineage-bearing branches.

The artifact includes:

- a closed event and effect model;

- a pure projector and explicit settlement scheduler;
- branch construction with parent/child identity, cut, shared prefix, continuation, inherited effects, and prefix hash;
- cut-level fork assessments and replay-grade computation;
- an ActiveGraph/bridge JSONL normalizer and batch validator;
- 44 passing tests, including FsCheck properties and named fixtures;
- seven language-neutral JSON conformance vectors and a JSON Schema;
- read-only SQLite export and actual-fork audit scripts;
- a findings log that records failed laws as they are discovered.

All F# projects treat warnings as errors. Pattern matches over closed domain unions are exhaustive. External effect execution is not linked into the kernel, so replay cannot accidentally call a model or tool through an injected client.

6.2 Property-based laws

FsCheck generates bounded event values, writer families, effect dimensions, and step bounds. Scheduler policies are enumerated rather than sampled because the executable model exposes four intended extremes. Properties include:

- deterministic projection and event-identity idempotence;
- exact-bound quiescence and bounded self-cycle divergence;
- convergence of isolated disjoint writers under all schedulers;
- schedule dependence for generated conflicting writers;
- identity, nested-collapse, projection, and suffix laws for forks;
- verdict matrices across generated footprint and replay-source values;
- grade-chain lattice laws and prerequisite enforcement.

FsCheck shrinking is useful for scalar values, but the semantically important counterexamples are named and checked in as JSON-backed fixtures. The fixtures include overlapping writers, read/trigger interference despite disjoint writes, a self-cycle, a discarded OneShot effect, an unresolved request, malformed effect transitions, and forged look-alike evidence types.

6.3 Real-log normalization

Validation accepts ordered JSONL. For SQLite-backed ActiveGraph stores, an export script uses `events.seq` as the ordering authority and emits one file per run. Source domain events that are not part of the closed effect or evidence vocabulary remain signals and are reported by type. The adapter does not infer effects from every `*.requested` event because real domains use names such as `round.requested`; only a closed list of known external request types is classified.

For an external request, footprint is read from explicit side-effect metadata. A write without an idempotency key is OneShot; an unknown tool footprint remains Unknown. Model and embedding requests default conservatively to OneShot. A response is Recorded only when the log contains an explicit response hash, output hash, or raw output from which the validator derives a content SHA-256. The derived hash certifies captured replay material; it is not a provider receipt or proof of external commitment.

Every source event is assigned a contiguous normalized sequence. Bridge events can produce multiple evidence facts; this is why the bridge normalized count exceeds its source count. The source event identity is retained for the first fact, and deterministic derived identifiers are used for additional facts. Duplicate or non-contiguous normalized identity remains a blocker.

6.4 All-cut enumeration

For a normalized log of length n , the validator evaluates cuts $0, \dots, n$. Prefix states are computed with one scan; environmental findings for suffixes are accumulated in reverse. The validator reports Sound, Conditional, and Unsound totals for ProjectionReplay, ExternalContinuation, and CounterfactualWorld, plus the number of runs containing an unsound counterfactual cut.

Actual ActiveGraph forks require a second audit because the child trace has lineage and store-level sequence differences. The SQL audit resolves the parent cut event, orders the parent prefix and child trace independently, and compares corresponding event ID, type, actor, payload, frame, cause, and timestamp. Child run ID and the database’s global autoincrement sequence are excluded by definition. The audit separately counts retained external requests and unresolved boundaries.

6.5 Reproducibility discipline

Evidence entries record repository, immutable revision when available, source path, artifact hashes, counts, parser profile, command, publication status, and limitations. Checked-in fixtures are hash-bound and explicitly marked synthetic or sanitized. Private and local runtime payloads are not copied into the repository merely to make a test convenient. Missing revision or source provenance is reported as a limitation rather than replaced with a hash-only claim of reproducibility.

7 Validation Against Runtime Artifacts

We evaluate the laws against two independent trace corpora. The first is a public, sealed Synthetic Players capsule containing an ActiveGraph SQLite store. The second is a local activegraph-bridge synthetic executive study with explicit boundary verification. Both contain synthetic experimental content; “real” here means that we analyze the actual runtime artifacts and their recorded failure modes rather than invented shape fixtures. We do not claim human behavioral validity or real-model performance for the bridge study.

7.1 Synthetic Players capsule

The public source is pinned to revision `82556ef9dae3`. The compressed engine store begins SHA-256 `7b7fdfe64b5b`, and the capsule checksum ledger begins SHA-256 `6d80f3b19b92`. The evidence ledger records the complete revision and digests. Running the public verifier reports:

CAPSULE VERIFICATION PASS — 4,919 archived Phase 3–5 runs verified (4,916 confirmatory + 3 legacy diagnostics).

The decompressed database contains 5,540 runs and 311,756 events. Event types include 36,251 `llm.requested`, 36,227 `llm.responded`, and 30,397 `decision.parsed` records. The difference between request and response counts is reflected rather than repaired by the adapter.

7.1.1 Replay grades

All 5,540 logs grade Observed, and zero are verified *from the run log alone*. This does not contradict the passing capsule verifier. The capsule attestation is external to each run’s event stream. A grading function that claims to depend on the log alone cannot silently import that attestation. This is a useful negative result: sealed-artifact reproducibility and self-describing run replayability are distinct properties.

7.1.2 All-cut results

Table 4 reports every cut. The projection total is $311,756 + 5,540 = 317,296$, because each run of length n has $n + 1$ cuts.

Property	Sound	Conditional	Unsound
ProjectionReplay	317,296	0	0
ExternalContinuation	160,076	120,969	36,251
CounterfactualWorld	150,230	24,172	142,894

Table 4: All-cut verdicts for 5,540 Synthetic Players logs.

Every projection cut is sound under the specified domain-projection observation. The 36,251 ExternalContinuation-unsound cuts equal the number of open/cut-through LLM requests. Conditional continuation cuts retain a committed OneShot oracle interaction and require serving the recorded result without another call. CounterfactualWorld is stricter: 4,923 runs have at least one Unsound cut because a discarded oracle interaction already occurred or a boundary is ambiguous.

The validator leaves domain and infrastructure types unclassified, including `behavior.started` and `decision.parsed`. It likewise leaves `infra.*`, `round.requested`, and `run.completed` unclassified. In particular, `round.requested` is not misclassified as an external request merely because its name ends in `requested`.

7.1.3 Observed forks

The SQLite run table contains 121 child runs from 41 distinct parents and no nested child in this sample. All forks cut at a `round.played` event. The audit compares 16,625 parent-prefix events with the beginning of their child traces and finds zero mismatches in event ID, type, actor, payload, frame, cause, or timestamp. Every cut event resolves. Run identity and the database’s global sequence are excluded because they are expected to differ.

None of the 121 retained prefixes contains a classified external request, and none contains an unresolved request. The observed forks therefore satisfy the current ExternalContinuation precondition for a domain-only cut family. This is strong evidence for cheap structural branching where the runtime actually forked. It is not evidence for an effect-boundary fork: the observed sample does not contain one.

7.2 activegraph-bridge executive study

The second artifact contains 12 synthetic vendor due-diligence cases evaluated under two deterministic offline mock strategies, for 24 runs and 2,592 source events. The manifest states that no real LLM or external provider was used and limits the claim to instrumentation feasibility. It records fresh-factory reconstruction and enables boundary verification.

The artifact files are hash-bound. Hash prefixes for the run store, manifest, metrics, and paired results are `7a38a1e8`, `da1cbc3b`, `9f9242e1`, and `f4d95dbb`, respectively; the evidence ledger records the complete hashes. However, the containing migration-lab directory has no Git commit or configured remote and all source files are untracked. The local result is content-addressed but not externally source-reproducible. We retain the finding and state this provenance defect plainly.

7.2.1 Decision and path equivalence

The paired artifact reports 66.6667% final-decision agreement and 0% ordered tool-path agreement. Eight of twelve pairs reach the same decision via different paths, yielding 66.6667% same-decision/different-path. This is not a model-quality result—both strategies are deterministic mocks. It is an instrumented witness that decision equivalence and trace equivalence answer different questions. A system that reports only agreement can hide total path divergence; a system that requires exact paths can reject decisions that are equal under a task-level observation.

7.2.2 Grades and cuts

Each run’s source events normalize to two additional evidence facts: fresh reconstruction and mediated boundary. The 2,592 source events therefore become 2,640 normalized events. All 24 logs grade Boundary and all 24 are verified from explicit log facts.

Property	Sound	Conditional	Unsound
ProjectionReplay	2,664	0	0
ExternalContinuation	1,992	288	384
CounterfactualWorld	0	288	2,376

Table 5: All-cut verdicts for 24 bridge logs.

All 2,664 projection cuts are sound. Every run contains a OneShot `submit_recommendation` effect. No cut is unconditionally Sound for CounterfactualWorld: before the commit, the effect lies in the discarded suffix; after the commit, continuation is Conditional on not executing it again. The result demonstrates why Boundary grade licenses effect-boundary operations but cannot promise a world in which discarded actions never happened.

7.3 Answers to the validation questions

The research brief posed three concrete validation tasks.

V1: compute grade distributions. Completed. The public capsule yields Observed for 5,540 logs because verification is external to the log; the bridge yields Boundary for 24 logs because the necessary evidence is explicit. The contrast supports log-derived grading rather than profile-derived labels.

V2: inspect real forks and their preconditions. Completed for the public capsule. All 121 observed forks preserve the compared prefix fields and retain no classified external request. No observed fork tests an oracle boundary, so that case remains open.

V3: test production confluence conditions. Partially completed. Source audit confirms that ActiveGraph imposes FIFO plus registration order, which can mask non-confluence. The property suite bypasses that order. A complete inventory of production behavior reads, writes, emissions, triggers, and relation-order dependencies was not available, so we cannot certify or falsify a general production non-interference contract.

Combined interpretation. The evidence supports deterministic projection and structurally correct observed prefix copying. It does not support an unrestricted claim that any historical cut is safely executable or environmentally counterfactual. Those stronger claims require effect-complete evidence and a property-specific precondition.

8 Related Work and Novelty Boundary

Reducer architectures. Mealy machines produce output and a next state from a current state and input [7]. Elm’s update loop maps a message and model to a new model and command [3]. Chassaing’s functional event-sourcing Decider separates decision and evolution and demonstrates composition in F# [2]. These precedents invalidate a novelty claim for the reducer shape. Our contribution begins where the runtime distinguishes replayable facts from externally executed effect requests and asks about specific fork cuts.

The title “Agent Algebra” was also abandoned because Burch, Passerone, and Sangiovanni-Vincentelli already use Agent Algebra for a general framework of concurrent models, refinement, composition, and conformance [1]. The present project is consequently named for the laws it checks rather than an algebra it does not originate.

Agent calculi and mechanized semantics. λ_A gives a typed lambda calculus for agent composition with oracle calls, bounded fixpoints, probabilistic choice, and mutable environments. It proves type safety and bounded termination in 1,519 lines of Coq with 42 theorems [5]. Its properties are primarily about composition-time well-formedness and bounded constructs. We do not duplicate that contribution; we study runtime event logs, schedule-sensitive settlement, and effect-boundary forks.

LLMbda models agent conversations, fork and clear operations, code generation, and labeled information flow. Its central result is a machine-checked termination-insensitive probabilistic noninterference theorem [4]. Our work does not claim noninterference or prompt-injection safety. Its use of conversation forks is related operationally, but our property of interest is whether an event-log cut licenses replay and an external-world claim.

Schlapbach formalizes Schema-Guided Dialogue and the Model Context Protocol in a process calculus and studies structural bisimilarity and expressiveness [9]. That is a protocol-level relation. We treat model and tool interactions as effect boundaries inside a runtime trace and do not compare protocol expressiveness.

Runtime substrates and governance. Agent libOS supplies a capability-controlled runtime with process identity, memory, skills, JIT tools, checkpoints, restore, fork, and commit [11]. It demonstrates operational mechanisms and a safety benchmark. We do not contribute a capability

system. Our narrower question is which algebraic and evidentiary conditions make a particular event-log fork sound for a named observation.

McCann’s Effect-Transparent Governance formalizes handler-mediated effect governance in Rocq and proves semantic preservation, expressive minimality, and decidability boundaries [6]. This directly motivates our decision to concede minimality. We use an effect-transparent boundary as an executable assumption, then investigate replay-source, lifecycle, and external-footprint evidence rather than proving governance expressivity.

AgentSpec lets users state runtime rules as triggers, predicates, and enforcement mechanisms and evaluates safety across code, embodied, and driving domains [10]. Its contribution is customizable runtime enforcement. Our laws do not enforce domain policy; they decide which replay or fork statements are licensed by a retained trace.

ActiveGraph. ActiveGraph makes the event log authoritative and presents deterministic replay, cheap forks, and lineage as architectural properties [8]. This paper is explicitly a follow-on. It preserves the fixed-log replay result, shows that reactive settlement needs separate termination and confluence conditions, and reframes cheap forking as a semantic claim indexed by projection, execution, continuation, or external-world equivalence. The public ActiveGraph implementation and its downstream artifacts supply the validation substrate unavailable to purely formal work.

What remains distinct. We are not aware, within the cited agent-calculus and runtime work, of an executable study that jointly (i) permutes runtime firing order, (ii) preserves negative confluence witnesses, (iii) partitions effect evidence along footprint/source/lifecycle dimensions, (iv) assesses every cut under multiple fork properties, and (v) checks those laws against an existing event-sourced runtime’s archived traces and observed forks. This is the bounded novelty claim for the draft; it should be revisited with a broader systematic search before submission.

9 Discussion

9.1 “The log replayed” is not one assertion

The empirical results expose a hierarchy of claims that are often collapsed in agent-runtime descriptions.

1. **Byte replay:** a serialized artifact is identical under a verifier.
2. **Trace replay:** the same ordered semantic events are retained.
3. **Projection replay:** folding the events yields an equivalent state under a chosen observation.
4. **Path replay:** the same model/tool/action sequence occurs.
5. **Decision equivalence:** a task-level conclusion agrees.
6. **Execution replay:** the run can be reproduced with zero live calls by serving deterministic or recorded effect results.
7. **Environmental equivalence:** the external world matches the world described by the retained prefix.

None of these relations automatically implies all the others. The bridge study has equal decisions in two thirds of pairs and equal ordered paths in none. The Synthetic Players capsule passes an external byte-exact verifier but its individual logs contain no verification attestation, so they remain Observed under a log-alone grade. Every prefix projects, but cuts through requests fail execution continuation. These are not contradictions; they are different predicates.

9.2 Cheap fork is semantic before it is fast

Sharing an immutable prefix is computationally cheap. The harder question is what the child is allowed to do next. A runtime that advertises `fork(k)` should specify at least:

- the observation under which parent prefix and child are equivalent;
- the ordering authority for the prefix;
- whether a cut may split a request from its outcome;
- how recorded results are served without re-execution;
- which retained external effects are inherited;
- whether the child receives an isolated/snapshotted environment or the already-mutated source environment;
- the treatment of external effects in a retrospectively discarded suffix.

Without these clauses, “cheap” describes storage cost while leaving semantic cost undefined. With them, the implementation can refuse unsafe cuts, require an idempotency key, request compensation, select a snapshot, or downgrade the claim to `ProjectionReplay`.

9.3 Deterministic ordering can conceal non-confluence

FIFO and registration order are useful engineering contracts. They make one execution repeatable and improve debuggability. But they can turn an accidental ordering dependency into a stable behavior, making ordinary replay tests pass while schedule independence remains false. This matters when a runtime later parallelizes handlers, changes relation storage, distributes a queue, or moves from one database backend to another.

The appropriate response need not be to demand full confluence. Some systems may intentionally make order semantic. The contract should then name the canonical order, preserve it in the log, and reject backends that cannot supply it. Systems that want schedule independence need a stronger behavior non-interference discipline. The current work distinguishes the choices but does not prescribe one universally.

9.4 The log should carry its own licenses

The gap between capsule verification and log-alone grade suggests a design principle: evidence needed to authorize later replay or fork should be retained with the run or its signed sidecar. A minimum receipt should include canonical sequence, run identity, parent and cut lineage, effect request identity, footprint annotation, outcome linkage, response content hash, commitment or failure evidence, reconstruction method, and verification result.

This does not require every event to be trusted merely because it is present. The event type must be recognized, the producer or signature may need authentication, and cross-event integrity must be checked. The present artifact enforces structural integrity and exact recognized types but does not implement cryptographic attestation. A future signed evidence capsule could make grade computation portable across implementations.

9.5 The graph may be a projection, but the scheduler remains real

The original minimality experiment asked whether an agent graph is fundamental or merely a projection over event-producing reducers. The present findings support part of that intuition: replayed domain state can indeed be treated as a projection. They also reveal what the reduction omits. Once multiple behaviors react, event and activation order become part of the operational semantics unless a confluence condition removes them. Once effects leave the process, the external world becomes a second state that the event projection can describe but not reverse.

Thus reducing the runtime to a pure fold is useful but incomplete. The deeper object is an event-producing transition system plus a scheduler, an effect interpreter, and evidence linking interpreter outcomes back to the log. The graph may be derived; the ordering and effect contracts cannot be wished away.

9.6 Operational recommendations

The results suggest concrete contracts for event-sourced agent runtimes:

1. Treat production order as an explicit scheduler policy and test at least event-order and activation-order perturbations.
2. Record behavior read, write, emission, and trigger dependencies if schedule independence is a goal; a write set alone is insufficient.
3. Separate replay source from external footprint. “Cached” does not mean “never happened” or “safe to repeat.”
4. Make request and terminal outcome a one-to-one integrity boundary. Refuse a cut through Requested or Unknown without an owner-selected recovery policy.
5. Give every fork API a property parameter or an equivalently explicit mode. Do not return a single unqualified **safe** Boolean.
6. Preserve verification and reconstruction evidence inside a portable, hash-bound run capsule.
7. Distinguish actual observed forks from every-cut counterfactual tests. The former show where users forked; the latter reveal untested hazards.

These recommendations are compatible with F#, C#, Python, Rust, or TypeScript. The conformance vectors intentionally encode events and expected verdicts in language-neutral JSON so another implementation can challenge the semantics.

10 Limitations and Threats to Validity

No mechanized proof. The propositions are encoded as executable laws and checked over finite generators and artifacts. A passing FsCheck property is not a proof over all behaviors, schedulers, or logs. This is a conscious genre choice, but it limits the strength of words such as “law.” Future work may mechanize selected definitions after the operational contracts stabilize.

No minimal confluence condition. We falsify general confluence and declared-write disjointness. We check a restricted isolated-writer family. We do not establish a necessary or sufficient general condition over reads, writes, emissions, triggers, and state domains. Calling the current restricted condition “the” restoring condition would overstate the result.

Limited scheduler space. The executable suite uses four deterministic scheduler extremes. They expose the preserved counterexamples but do not enumerate every interleaving for an arbitrary behavior set. Relation ordering is discussed from source audit but not modeled as a separate production activation dimension.

Adapter assumptions. Real-log classification relies on explicit event names and payload fields. The adapter is deliberately conservative, but mistakes remain possible. Model calls are treated as OneShot for cost/oracle semantics; another property might classify a read-only deterministic local model differently. A derived content hash proves captured bytes, not provider authenticity, commitment, or idempotence.

Whole-log malformedness policy. The current assessor treats structural malformedness anywhere in the source log as a blocker for every cut, including projection. This is fail-closed and appropriate for the evidence study, but it is stricter than a theorem about an independently authenticated well-formed prefix. The projection suffix law is therefore stated only for well-formed ordered logs.

Synthetic content. The Synthetic Players capsule contains archived experimental runs, but the present analysis concerns runtime traces rather than human ground truth. The bridge study uses deterministic offline mocks and explicitly makes no real-model performance claim. Its agreement numbers illustrate equivalence relations, not predictive validity.

Fork coverage. The 121 observed forks are useful positive evidence, but all cut at `round.played` and retain no classified external request. No observed fork in the public capsule crosses a recorded oracle or irreversible effect boundary. The difficult cases are exercised by all-cut analysis and fixtures, not by an actual public fork with a no-reexecution receipt.

Bridge provenance. The bridge artifact is locally content-addressed but lives in an unborn Git repository with no remote. A fresh external reviewer cannot reconstruct the source revision. The result should remain a draft finding until the artifact and generator are committed or published. This limitation is particularly important because provenance is part of the paper’s thesis.

Production behavior inventory. The ActiveGraph source establishes FIFO and registration-order scheduling, but we did not recover a complete deployed behavior dependency inventory. We therefore cannot say whether production behaviors satisfy a future non-interference condition. Static source inspection of the scheduler is not production confluence validation.

No authorization or information-flow result. Effect descriptors classify replay and environmental conditions; they do not grant capabilities or prove that a caller is authorized. The artifact also does not track secrecy labels or prove noninterference. Those topics are covered by distinct systems and calculi and remain explicit non-goals.

No performance benchmark. The all-cut implementation is efficient enough for the reported corpora, but we make no throughput, storage, or latency claim. “Cheap” in this paper is analyzed semantically. A separate benchmark would be needed for production performance.

Size target. The core-law implementation is 1,393 lines rather than the desired roughly 1,000. The transition/settlement boundary remains 594 lines. The additional fork and grade diagnostics improved fail-closed behavior, but the overage may still reduce readability. Refactoring should preserve named findings and law coverage rather than optimizing only for a line count.

11 Conclusion

Deterministic projection of a fixed log is valuable, but it is the beginning of an agent-runtime replay contract rather than the end. Reactive settlement can diverge, a stable production order can mask non-confluence, and disjoint declared writes do not exclude schedule-sensitive reads or trigger interference. A fork can preserve its trace prefix while failing to reproduce execution or the external world.

The executable specification makes those distinctions checkable. Effect footprint, replay source, and lifecycle answer separate questions. Fork safety is indexed by an observation: projection, strict execution replay, continuation in a prefix-reconstructed environment, or counterfactual-world equivalence. Replay grades summarize retained evidence, while cut-level assessment decides whether a particular branch meets the property-specific preconditions.

The runtime artifacts support both positive and negative conclusions. Across 5,540 public capsule logs, every one of 317,296 projected prefixes is sound under the stated projection observation. Across 121 observed forks, 16,625 shared-prefix events show zero structural mismatches modulo expected lineage fields, and the real cuts avoid external requests. Yet 36,251 hypothetical cuts split model requests and are unsound for continuation, while thousands of discarded oracle interactions prevent an unchanged-world claim. In a second study, 66.7% decision agreement coexists with 0% path agreement.

The practical conclusion is concise: replay is a family of assertions, and a fork is sound only relative to one of them. Event-sourced agent runtimes should record the evidence that licenses each assertion, expose ordering assumptions, and refuse to turn a cheap prefix copy into an unqualified promise about execution or the world.

References

- [1] Jerry R. Burch, Roberto Passerone, and Alberto L. Sangiovanni-Vincentelli. Notes on agent algebras. Technical Report UCB/ERL M03/38, University of California, Berkeley, 2003.

- [2] Jérémie Chassaing. Functional event sourcing decider. Online article, 2021.
- [3] Elm Project. The elm architecture. An Introduction to Elm.
- [4] Zac Garby, Andrew D. Gordon, and David Sands. The llmbda calculus: Ai agents, conversations, and information flow. *arXiv preprint arXiv:2602.20064*, 2026.
- [5] Qin Liu. λ_a : A typed lambda calculus for llm agent composition. *arXiv preprint arXiv:2604.11767*, 2026.
- [6] Alan L. McCann. Effect-transparent governance for ai workflow architectures: Semantic preservation, expressive minimality, and decidability boundaries. *arXiv preprint arXiv:2605.01030*, 2026.
- [7] George H. Mealy. A method for synthesizing sequential circuits. *Bell System Technical Journal*, 34(5):1045–1079, 1955.
- [8] Yohei Nakajima. The log is the agent: Event-sourced reactive graphs for auditable, forkable agentic systems. *arXiv preprint arXiv:2605.21997*, 2026.
- [9] Andreas Schlapbach. Formal semantics for agentic tool protocols: A process calculus approach. *arXiv preprint arXiv:2603.24747*, 2026.
- [10] Haoyu Wang, Chris Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.
- [11] Yingqi Zhang. Agent libos: A runtime substrate for capability-controlled self-evolving llm agents. *arXiv preprint arXiv:2606.03895*, 2026.